

Four Hands-On Builds for AI Accuracy Engineering

Four independent, end-to-end applications built on an Azure virtual machine with GitHub Copilot — each one a working manufacturing system, each one designed to make a Day 1 concept impossible to forget.

Companion sessions	Module 1 — Accuracy to system reliability · Module 2A — Evaluation foundations · Module 2B — Hypothesis testing · Module 3 — Error taxonomy and root cause
Companion demonstration	Demo 2 — Predictive Maintenance and Anomaly Triage (delivered Days 1 and 4)
Format	Teams of four. Tasks 1–3 are fully self-contained. Task 4 builds on Task 2, but a substitute baseline model is quick to produce if you need one.
Toolchain	Azure Linux VM · GitHub Copilot (agent mode) · Python 3.11 · Streamlit · Azure OpenAI chat deployment
Every task ends with	A running dashboard on a URL the facilitator can open, plus a one-screen verdict your team defends aloud to the room.

The rule that governs all four tasks. You are not being scored on whether your model is good. You are being scored on whether you can prove what your system does and does not do — with a number, an interval, and a stated cost. A team that reports 0.62 recall with an honest confidence interval and a cost-justified threshold beats a team that reports 0.99 accuracy every single time.

Task	Build	Where it runs	What it makes undeniable
1	SPC & Automatic Feedback Console Machined component height	Lab 1	A control chart is a classifier. Its alarm rules have a false-positive rate, a detection lead time, and a cost.
2	Failure Prediction Operating-Point Studio Real equipment telemetry	Lab 2	99% accuracy with near-zero recall. Target leakage. The threshold that alarm-fatigue actually permits.
3	Failure Taxonomy & Hypothesis Backlog Console Real maintenance work orders	Lab 3	Where two engineers disagree, no model can learn a boundary. Two categories carry most of the cost.
4	Production Readiness Review Overnight, scored	Day 1 assignment overnight, scored	Proving a change helped: paired test, effect size, power, drift, calibration, and a defensible ship decision.

Tasks 1–3 run inside the Day 1 lab sessions. Task 4 is the scored assignment and replaces nothing — it consolidates the first three into the seven deliverables listed on the Day 1 assignment slide. Readouts are given at the start of the session following each lab, so they do not consume lab time.

Before you start — environment and ground rules

One VM per team

Set this up before the first lab begins — during the opening module, or the evening before. A team still installing packages when the lab starts has lost a large part of it.

1 · The virtual machine

Size	Standard_D4s_v5 — 4 vCPU, 16 GB RAM. Sufficient for every task in this pack. No GPU is required on Day 1.
Image	Ubuntu Server 24.04 LTS
Disk	64 GB Premium SSD
Networking	SSH (22) and Streamlit (8501) inbound, source restricted to the training-room public IP . Never leave 8501 open to the internet.
Naming	vm-acday1-team<n> — the facilitator opens these by name during the readout.

```
# From Azure Cloud Shell — one command per team
az group create -n rg-acday1 -l westeurope

az vm create -g rg-acday1 -n vm-acday1-team1 \
  --image Ubuntu2404 --size Standard_D4s_v5 \
  --admin-username azureuser --generate-ssh-keys

# Restrict to the training-room egress IP. Do NOT use `az vm open-port` — it opens 8501 to *.
az network nsg rule create -g rg-acday1 --nsg-name vm-acday1-team1NSG \
  -n allow-streamlit --priority 900 --access Allow --protocol Tcp \
  --destination-port-ranges 8501 --source-address-prefixes <training-room-ip>/32
```

2 · The toolchain

```
ssh azureuser@<public-ip>
sudo apt update && sudo apt install -y python3-pip python3-venv git
python3 -m venv ~/.venv && source ~/.venv/bin/activate
pip install streamlit pandas numpy scipy scikit-learn plotly statsmodels \
  openai python-dotenv pyarrow fpdf2 jinja2
mkdir -p ~/task1 ~/task2 ~/task3 ~/task4 && cd ~/task1
```

Editing. Use VS Code with the Remote-SSH extension from your laptop, connected to the VM, with GitHub Copilot enabled. Agent mode must be permitted by organisation policy — confirm this *before* Day 1, not on the day. Run `streamlit run app.py --server.address 0.0.0.0` and open `http://<public-ip>:8501`.

3 · Azure OpenAI

Used in every task for one job only: turning a measured result into language a manufacturing manager reads. It never computes a metric, never chooses a threshold, and never decides anything. If your Azure OpenAI call fails, every task still completes — the language panel simply falls back to a template.

```
# ~/.env — ask for one deployment per team; a shared one will rate-limit (see below)
AZURE_OPENAI_ENDPOINT=https://<your-resource>.openai.azure.com/
AZURE_OPENAI_API_KEY=<key>
AZURE_OPENAI_DEPLOYMENT=gpt-4o-mini
AZURE_OPENAI_API_VERSION=2024-10-21
```

```

from openai import AzureOpenAI; import os
from dotenv import load_dotenv; load_dotenv()

client = AzureOpenAI(
    azure_endpoint=os.environ["AZURE_OPENAI_ENDPOINT"],
    api_key=os.environ["AZURE_OPENAI_API_KEY"],
    api_version=os.environ["AZURE_OPENAI_API_VERSION"])

def narrate(system, facts):
    r = client.chat.completions.create(
        model=os.environ["AZURE_OPENAI_DEPLOYMENT"], temperature=0.2,
        messages=[{"role": "system", "content": system},
                  {"role": "user", "content": facts}])
    return r.choices[0].message.content

```

Watch for — the failure mode that will bite two teams today. Twenty-five people querying one Azure OpenAI deployment at once will hit the tokens-per-minute limit regardless of budget. Wrap every call in a try/except that falls back to a plain-text template, and do it *when you write the call*, not after it breaks during your readout. This is itself a Day 1 lesson: a system's reliability is not its model's accuracy.

4 · Repository hygiene

`git init` in each task folder and commit after every movement. Task 4 asks you to compare two model configurations and prove one is better; that comparison is far easier when both still exist. Commit messages become your experiment lineage, which is the last of the seven assignment deliverables.

5 · Working with Copilot — the discipline that separates the top teams

Copilot in agent mode will build any of these applications quickly. That is exactly why the tasks are designed the way they are: speed of construction is not the constraint, and the teams who finish first are usually the teams who have not noticed what is wrong.

Three rules for Copilot use in these labs.

1. **Never accept a metric you did not ask for.** Copilot will volunteer accuracy. On imbalanced manufacturing data accuracy is close to meaningless. Ask for the metric you defined, and delete the rest.
2. **Ask it to argue against itself.** After it produces a pipeline, prompt: *"List every way this evaluation could be dishonest. Check leakage, splitting, and whether any feature encodes the outcome."* This prompt has found the planted trap in every pilot run of this material.
3. **Read the split.** Whatever else you skim, read the lines that create train and test sets. More PoCs die there than anywhere else in the file.

6 · How your work is scored — identical for all four tasks

Criterion	Points	What full marks looks like
The system runs	15	Facilitator opens the URL, it loads, it responds. No screenshots.
Metric set is defined and defended	20	Three to five metrics, each with a target and a stated reason it is the right one for this decision.
Uncertainty is quantified	15	Every headline number carries an interval. Bootstrap is acceptable and preferred.
The failure mode is demonstrated	20	The wrong number and the corrected number are shown together, with the mechanism explained in one sentence a non-specialist follows.
Cost drives the operating point	15	The threshold is derived from stated costs, not chosen at 0.5, and the choice is shown on a curve.
The verdict is actionable	15	A plant or service manager could read one screen and know what to do. No raw metric dumps.

Automatic deduction: minus 10 for any headline figure presented without an interval; minus 10 for any threshold set at 0.5 without justification. These are the two habits Day 1 exists to break.

Task 1 — SPC and Automatic Feedback Console

Lab 1

MODULE 1

THRESHOLDS

COST OF ERROR

ERROR BUDGET

ALEATORIC VS EPISTEMIC

The scenario

A machining cell finishes the mating face of a compressor element end cover. The controlled characteristic is **face height, nominal 88.000 mm, tolerance ± 0.100 mm**. Five parts are gauged every fifteen minutes across three shifts. Today the cell runs unattended overnight, and the quality engineer sees the data at 07:00 the next morning — which means the system, not a person, has to decide when something has gone wrong and say so in language a shift leader can act on.

You will build that system: it generates the measurement stream, charts it, decides, and writes the handover note. Then — and this is the part that matters — you will treat your own alarm logic as a classifier and measure how good it actually is.

Why this is a Day 1 task and not a quality-department task. A control chart is a binary classifier that fires on every subgroup. It has a false positive rate, a detection delay, and a cost per decision — exactly the three quantities Module 1 says a single accuracy figure cannot express. The reason it belongs at the start of an AI course is that the room already believes in it, so nobody argues about the framework while learning it.

Step 1 · The process simulator

Write `process_sim.py` to this exact specification. Every team uses the same specification so the leaderboard is comparable.

Element	Specification
Subgroups	192 subgroups of $n = 5$ parts (two production days, 15-minute sampling). 960 measurements total.
In-control process	Height $\sim \text{Normal}(\mu = 88.000 \text{ mm}, \sigma = 0.025 \text{ mm})$. Note this gives $C_p = 1.33$ exactly — a process that just passes the usual gate.
Special cause A — tool wear	From subgroup 60, μ increases by 0.0008 mm per subgroup for 40 subgroups, then a tool change resets it. A slow ramp: no single point looks alarming.
Special cause B — shift step	At subgroup 110, μ steps to 88.017 mm and stays there for 25 subgroups. That is only 0.68σ on an individual part and 1.5σ on the subgroup mean — a plain 3σ rule needs about 14 subgroups to notice it.
Special cause C — fixture slip	Subgroups 150–152 only: σ triples to 0.075 mm, μ unchanged. Spread changes without the mean moving.
Measurement system	Round every reading to 0.01 mm (gauge resolution). Independently, 1% of readings come from a mis-zeroed gauge and carry a +0.050 mm offset.
Ground-truth log	The generator must also emit <code>truth.csv</code> with three columns per subgroup: whether the process was genuinely in control, which cause was active, and <code>gauge_fault</code> — whether that subgroup contained a mis-zeroed reading. You cannot score a detector without this.
<code>--clean</code> mode	A flag that disables all three special causes and the gauge fault, and accepts <code>--subgroups</code> . You need 2,000 clean subgroups in Step 4 to measure a false-alarm rate at all.
Seeds	Develop on <code>seed=42</code> . A second seed will be announced partway through the lab — see Step 6.

COPILOT — STARTING PROMPT

Create process_sim.py. It simulates a machining process measuring end-cover face height, nominal 88.000 mm, spec 87.900-88.100 mm. Generate 192 subgroups of 5 parts. Baseline Normal(88.000, 0.025). Inject exactly three special causes:
(A) linear mean ramp +0.0008 mm/subgroup from subgroup 60 for 40 subgroups;
(B) mean step to 88.017 at subgroup 110 lasting 25 subgroups;
(C) sigma x3 at subgroups 150-152 with mean unchanged.
Then round all readings to 0.01 mm and add a +0.050 offset to a random 1% of readings.
Write measurements.csv (subgroup, part, height, timestamp) and truth.csv (subgroup, in_control bool, cause label, gauge_fault bool).
Accept --seed, --subgroups and a --clean flag that disables all causes and the gauge fault.
Do not add any other behaviour.

Step 2 · Charts and capability

- $\bar{X}$ and R charts with control limits from the standard constants (A_2 , D_3 , D_4 for $n = 5$).
- **Nelson rules 1–8** as a rule engine that returns, for each subgroup, the set of rules that fired.
- C_p , C_{pk} , P_p , P_{pk} — and a **bootstrap 95% confidence interval on C_{pk}** . Bootstrap over the **200 parts in the frozen baseline window** (subgroups 1–40) only; capability computed across out-of-control periods is meaningless. 2,000 resamples, parts drawn with replacement.

Trap 1 — the one Copilot will hand you. Ask for control limits and you will almost certainly get limits computed from all 960 measurements. But 68 of those 192 subgroups are out of control — 40 from cause A, 25 from B, 3 from C. Contaminated data inflates the limits, the limits then widen far enough to contain the very faults you are hunting, and your chart declares the process healthy.

Control limits must be estimated from a verified in-control baseline window — use subgroups 1–40 — and then frozen. Show both versions on your dashboard side by side. The difference is the entire lesson.

Trap 2 — specification limits are not control limits. If any part of your code compares a subgroup mean against 87.900 / 88.100, delete it. Specification limits describe what the customer will accept. Control limits describe what this process does when nothing is wrong. Confusing them is the most common error in industrial SPC and it is directly analogous to confusing a business target with a model's operating point.

Step 3 · The interval that changes the answer

Report C_{pk} with its bootstrap interval and compare it against the customary 1.33 acceptance gate. The approximate 95% half-width is $1.96 \times \sqrt{1/(9n) + C_{pk}^2/(2(n-1))}$ — roughly **± 0.14 on your 200 baseline parts, ± 0.06 on 1,000**. Your bootstrap should land near ± 0.14 ; if it does not, you have a bug worth finding. Note that the process is centred with $C_p = 1.33$ by construction, so the true C_{pk} is 1.33 — anything you observe above it is sampling noise, which is precisely the point.

The sentence to put on your dashboard. "Observed $C_{pk} = 1.35$, 95% CI 1.21 to 1.49 on 200 parts. This process cannot be shown to meet a 1.33 requirement, and it cannot be shown to fail it either. At 1,000 parts the interval is still 1.29 to 1.41 — it *still* straddles the gate. Separating 1.35 from 1.33 with confidence would take on the order of ten thousand parts."

The honest conclusion is therefore not "measure more". It is that **a process designed at $C_p = 1.33$ can never be demonstrated to clear a 1.33 gate** — the gate or the process has to change. That is a power argument, made with calipers, and it is exactly the argument that governs whether your evaluation set can detect the model improvement you are hoping for.

Step 4 · Score your alarm logic as a classifier

You have `truth.csv`. Use it. For each configuration of enabled Nelson rules, compute:

Measure	Definition on this data
False alarms per shift	Alarms raised during genuinely in-control subgroups, scaled to a 32-subgroup shift.
ARL ₀ (in-control)	Mean subgroups between false alarms. Measure this on a 2,000-subgroup <code>--clean</code> run , not on the 192-subgroup production run — that run contains only ~124 in-control subgroups and most teams will observe zero false alarms and an undefined ARL ₀ .
Detection lead, per cause	Subgroups from onset of A, B and C to the first alarm. Report all three separately — they behave completely differently.
Missed causes	Any injected cause that produced no alarm before it ended.
Measurement-system alarms	Alarms occurring in subgroups flagged <code>gauge_fault</code> . Count these in a third bucket — they are neither false alarms nor detections. They are the gauge failing, not the process, and confusing the two is one of the nine failure categories from Lab 1.

Published reference points to sanity-check against: a plain 3σ chart has $ARL_0 \approx 370.4$. Adding just the "2 of 3 beyond 2σ " rule drops it to ≈ 225 ; "4 of 5 beyond 1σ " to ≈ 166 ; "8 in a row on one side" to ≈ 153 . Those three supplementary rules together with the basic 3σ rule land near **92** — a false alarm roughly four times as often as the plain chart alone. Measure your own on the clean run; do not quote these.

Step 5 · Make cost choose the rule set

Event	Cost	Basis
False alarm — line stopped and investigated	€ 180	25 minutes of cell downtime plus inspection labour
Out-of-control production, per part	€ 14	Rework or scrap of parts made while undetected
Escape to assembly — <i>replaces</i> the per-part cost when a cause runs undetected to its end	€ 2,400	Teardown, containment and rebuild. Charge either the per-part cost <i>or</i> this, never both.

Enumerate rule subsets, compute expected cost per simulated run for each on the development seed, and plot **expected cost against rule configuration**. Your dashboard must state which rule set you selected and why — in euros, not in p-values.

Step 6 · The held-out seed

Deeper detail — and the reason this step exists. Partway through the lab the facilitator announces a second seed, without warning. Re-run your selected rule set on it, unchanged, and report the cost. The leaderboard uses **only the second seed**. Teams that tuned aggressively against the first will see their cost rise, sometimes sharply. This is tuning leakage — the seventh item on the leakage checklist in Module 3 — reproduced in fifteen lines of code inside a single lab. When Module 2A says the test set is held out and the validation set is where you iterate, this is the thing it is protecting you from.

Step 7 · The dashboard and the handover note

One Streamlit page. Top to bottom: **verdict banner** (green / amber / red with the active cause named) · $\bar{X}$ and R charts with violation markers · capability panel with the C_{pk} interval · violation log naming which rule fired at which subgroup · cost ledger · **shift handover note**.

COPILOT — THE AZURE OPENAI NARRATION CALL

Add a function `shift_note(violations, cause_guess, cost_summary, cpk_ci)` that calls Azure OpenAI and returns a 5-sentence shift handover note for a machining cell leader. System prompt: "You are writing a shift handover note for a machining cell leader at a compressor manufacturing plant. Use only the facts supplied. State what the chart detected, at what time, which rule fired, the most likely physical cause, and the single recommended action. Never invent a measurement. If no rule fired, say the process is in control and stop." Wrap in `try/except`; on any failure return a formatted template string using the same facts.

Watch for. The model must translate, never conclude. If your prompt lets it decide whether the process is capable, you have moved the decision from a computed interval to a language model — and you will not be able to defend that in the readout. Facts in, sentences out.

Acceptance criteria

- ☐ Dashboard runs on the team VM and the facilitator can open it.
- ☐ Control limits are computed from subgroups 1–40 only, and the all-data version is shown beside it for comparison.
- ☐ C_{pk} is never shown without its bootstrap interval.
- ☐ All three injected causes are reported with individual detection lead times — including any that were missed.
- ☐ The selected rule set is justified in euros of expected cost.
- ☐ Cost on the held-out seed is reported, and the change from the development seed is stated out loud.

Defend your result

"Our rule set costs € ____ per two-day run on data it has never seen. It detects a step shift in ____ subgroups and a wear ramp in _____. It misses _____. We accept that miss because _____."

Stretch, if you finish early. Add an EWMA chart ($\lambda = 0.2$) beside the Shewhart chart and compare detection lead on cause B, the small sustained step. EWMA should find it several subgroups sooner. Then explain to the room why the plant does not simply switch to EWMA everywhere — the answer is about who has to interpret the chart at 03:00, which is a system-reliability argument, not a statistical one.

Task 2 — Failure Prediction Operating-Point Studio

Lab 2

MODULE 2A

CLASS IMBALANCE

LEAKAGE

BOOTSTRAP CI

ALARM FATIGUE

REAL DATA

The scenario

Atlas Copco has roughly **1,200 installed machines under a service contract**, each scored once an hour by a failure-prediction service. Six field service engineers cover the region. Your team has been handed a model that reports very high accuracy and asked one question: **can we switch it on?**

By the end of this lab you will have an answer, a number, an interval on that number, and an operating point defended in alerts per engineer per day.

The data — real, and you will verify it yourself

Dataset	AI4I 2020 Predictive Maintenance Dataset , UCI Machine Learning Repository (ID 601). Synthetic but industrially faithful — it reflects a real milling machine process and is the standard public benchmark for this problem.
Size	10,000 rows, 14 columns.
Label	Machine failure — approximately 3.4% positive . Confirm the exact count in your first cell; do not take this sheet's word for it.
Sensors	Air temperature [K], process temperature [K], rotational speed [rpm], torque [Nm], tool wear [min], and a product quality grade L / M / H.
Also present	Five columns named TWF , HDF , PWF , OSF , RNF . Read Step 3 before you decide what to do with them.

```
wget https://archive.ics.uci.edu/static/public/601/ai4i+2020+predictive+maintenance+dataset.zip
unzip ai4i+2020+predictive+maintenance+dataset.zip
python -c "import pandas as pd; d=pd.read_csv('ai4i2020.csv'); \
print(d.shape); print(d['Machine failure'].value_counts()); \
print(d[['TWF','HDF','PWF','OSF','RNF']].sum())"
# If the VM has no egress to this host, the facilitator has ai4i2020.csv on the share.
```

Step 1 · Establish the dishonest baseline

Write a model that predicts "no failure" for every row. Report its accuracy. Put that number, in large type, at the top of your dashboard and leave it there for the rest of the session.

Say it aloud in your team before moving on. "Our system is 96.6% accurate, has never once predicted a failure, and would have prevented nothing." That sentence is the reason Day 1 exists. Everything you build for the next fifty minutes is an argument against the number you just wrote down.

Step 2 · A real model, and the confusion matrix

Stratified 70/30 split. Logistic regression and a gradient-boosted tree. At the default 0.5 threshold, report **accuracy, precision, recall and F1 on the failure class only**, plus the full confusion matrix. Expect accuracy above 0.97 and recall that will embarrass the model.

Do not fix this with SMOTE yet. Someone on your team will suggest it almost immediately. Write it on your dashboard as backlog hypothesis 1 and move on. In Step 6 you will discover that threshold selection — which changes no training at all — recovers most of what you wanted, and you will have learned something about where effort belongs.

Step 3 · The planted defect

Build a second model that includes `TWF`, `HDF`, `PWF`, `OSF` and `RNF` as input features. It will score close to perfect. Celebrate briefly, then work out why.

Target leakage, exactly as defined in Module 3. Those five columns are not predictors of failure — they are the *failure modes*, recorded at the moment the machine failed. The documentation states that `Machine failure` is their logical OR. You have handed the model the answer and it has dutifully returned it. In production the columns do not exist at prediction time; they are written afterwards, by the engineer, in the service report.

The industrial version of this is not obvious and it is everywhere. "Parts issued from stores", "work order closed", "downtime hours" — every one of them is a consequence of the failure, every one of them is sitting in the maintenance database looking exactly like a feature. Run Module 3's question over every column you keep: *at what moment in the real process does this value become known?*

Show both models side by side on the dashboard, permanently, labelled "**with leakage**" and "**honest**". It is the single most valuable screen you will build today.

Step 4 · Feature representation beats model choice

Now do the legitimate version. The three quantities below are computable entirely from sensors available before failure, and they encode the actual physics of the process:

Derived feature	Formula	Physical meaning
Mechanical power	<code>torque × 2π × rpm / 60</code> [W]	Power draw. The dataset's power-failure mode is defined by power leaving a working band.
Temperature difference	<code>process_temp - air_temp</code> [K]	Heat rejection. A small difference at low speed means heat is not being carried away.
Overstrain product	<code>tool_wear × torque</code> [min·Nm]	Accumulated mechanical strain on the tool.

Retrain the honest model with these three features added. Record the change in recall. Then record how much you gained in Step 2 by switching model family.

The finding to report. Three lines of arithmetic derived from process knowledge will outperform the change of model architecture, usually by a wide margin. That is Module 3's data-centric argument, produced by your own hands rather than asserted from a slide — and it is the reason "get more data" and "try a bigger model" are the two most expensive wrong answers in industrial AI.

Step 5 · Uncertainty, honestly

The test split contains roughly 100 failures. Recall computed on 100 positives is a noisy number. Bootstrap the test set 2,000 times and report **recall with a 95% interval**. Then answer, on the dashboard: *is the improvement in Step 4 larger than the interval, or inside it?*

Deeper detail. Bootstrap the *paired difference* — resample the same test rows once, score both models on that resample, take the difference, repeat. Then check whether the interval on the difference contains zero. Two overlapping individual intervals do not imply no difference; the paired interval is the one that answers the question, and it is the same argument that makes the paired designs in Module 2B worth the trouble.

Step 6 · The alarm budget — the constraint that decides everything

Every machine in the fleet is scored once a day. That is **1,200 predictions per day**. Six engineers. The service organisation will read **five alerts each** before it stops reading them — a hard ceiling of **30 alerts per day**.

First, correct the prior — because your training data lies about it

The benchmark is 3.4% positive. A real installed base is nothing like that: across 1,200 contracted machines, roughly **two per day** develop degradation worth dispatching an engineer for. That is a base rate of **0.17%**, twenty times rarer than your training set.

This is label shift, and it is not a detail. Module 2B lists three kinds of drift; this is the second — the class balance differs between where you trained and where you deploy. Your model's *ranking* of machines still transfers. Its *probabilities* and every threshold you set on them do not. Recompute precision at the deployment prior before you quote a single number to the business:

$\text{precision} = \text{TPR} \cdot \pi / (\text{TPR} \cdot \pi + \text{FPR} \cdot (1 - \pi))$ with $\pi = 0.0017$, not 0.034.

Then do the arithmetic that ends the conversation

Quantity	Value
Predictions per day (fleet × daily scoring)	1,200
Genuine degradations per day	≈ 2 ($\pi = 0.17\%$)
Alerts the organisation will read per day	$6 \times 5 = 30$
False alerts the budget therefore allows	$30 - 2 \cdot \text{recall} \approx 28$
Maximum tolerable false positive rate	$28 / 1,198 \approx 2.3\%$
Best achievable precision at full budget	$2 / 30 \approx 6.7\%$

Read the last row again, then put it on your dashboard in large type. Even if your model is perfect — catching every degradation, at the maximum alert volume this organisation can absorb — **fourteen out of every fifteen alerts an engineer opens will be nothing.** That is not a modelling failure. It is arithmetic, forced by a rare event and a finite number of engineers, and no amount of accuracy escapes it. This calculation should be done in week one of a predictive-maintenance project. It is usually done in month nine, by which time three people have been hired.

Now build the studio

Plot precision–recall, mark the operating point at $\text{FPR} \leq 2.3\%$, and report the recall you actually get there with its bootstrap interval. Build an **alert volume calculator**: a threshold slider showing, live, alerts per engineer per day, recall, precision *at the deployment prior*, and expected cost.

Cost input	Value
Unplanned stoppage on a customer installation	€ 9,500
Planned intervention when the warning was correct	€ 1,200
Engineer dispatched, nothing found	€ 700
Alert reviewed and dismissed from the desk	€ 35

Watch for. The threshold that minimises expected cost and the threshold the alarm budget permits are usually different. When they are, the honest deliverable is not a threshold — it is a sentence: "at the alert volume this organisation can absorb we detect ___% of degradations; to reach the cost-optimal point we would need ___ more engineers, or a triage step that dismisses alerts for € 35 instead of dispatching for € 700." That second option is almost always the right answer, and you will only find it because you did the arithmetic.

Step 7 · Audit the ground truth, and find the ceiling

Two checks, four lines of pandas between them.

Check one — does the label obey its own definition? Compare `Machine failure` against the logical OR of the five mode columns, in both directions. Count the rows where they disagree.

```
d['or_modes'] = d[['TWF', 'HDF', 'PWF', 'OSF', 'RNF']].max(axis=1)
print(pd.crosstab(d['Machine failure'], d['or_modes']))
print(d['RNF'].sum(), "RNF rows in the file") # compare with the documentation
```

What you will find, and why it matters more than your model. The label and the mode columns **do not agree**. There are rows labelled as failures where every failure mode is zero, and the number of random-failure rows in the file does not match the number the documentation describes. This is a public benchmark cited in hundreds of papers. Nobody checked.

This is **weak ground truth** — Module 3, section 3.3 — in the most quotable form available. Report the disagreement count on your dashboard and state the consequence in one sentence: no model can be more consistent than the labels it learns from, so this number is a hard ceiling on anything you build, and it was sitting in the first four lines of the file.

Check two — the irreducible part. **RNF** is a random failure, generated with a fixed probability and unrelated to any sensor value. Nothing in the feature set predicts it, at any sample size.

This is aleatoric uncertainty, in a dataset you can open. More data will not help. A bigger model will not help. The correct engineering response is to quantify it, exclude it from the improvement target, and stop spending money on it. Module 1 makes this distinction abstractly; here you can point at the rows.

Step 8 · The dashboard

Four panels: **fleet status** (machines ranked by risk) · **honest vs leaking** comparison · **operating point studio** (PR curve, threshold slider, live alert volume, cost) · **service advisory** — an Azure OpenAI note per top-ranked machine naming the sensor pattern driving the score and the recommended action, generated only from values you pass it.

Acceptance criteria

- ☐ The 96.6%-accuracy trivial baseline is visible on the dashboard.
- ☐ Leaking and honest models are shown together, with the leakage explained in one sentence.
- ☐ Recall carries a bootstrap 95% interval everywhere it appears.
- ☐ Gain from derived features is compared against gain from changing model family.
- ☐ The chosen threshold is stated in alerts per engineer per day, and precision is quoted at the deployment prior, not the training prior.
- ☐ Label-definition disagreements are counted, and the irreducible random subset is excluded from improvement targets.

Defend your result

"At five alerts per engineer per day we catch ____% of degradations, 95% CI ____ to ____, and ____ out of every ____ alerts is a real one. The leaking model claimed ____%. The largest honest gain came from ____, not from the model. We recommend switching it on / not switching it on because ____."

Leaderboard: highest recall on the failure class at a false positive rate of 2.3% or below, with the bootstrap interval reported. A team reporting recall without an interval does not place. Tie-break: the team whose stated precision at the deployment prior is closest to what their own numbers imply.

Task 3 — Failure Taxonomy and Hypothesis Backlog Console

Lab 3

MODULE 3

INTER-ANNOTATOR AGREEMENT

PARETO

SLICE ANALYSIS

COST WEIGHTING

REAL DATA

The scenario

The installed-base service organisation closes thousands of work orders a year, each carrying a few words of free text written by an engineer standing next to a machine. Somewhere in that text is the ranked list of what is actually going wrong — and nobody has ever counted it, because counting it requires agreeing what the categories are.

Your team will build the console that does the counting, and will discover early on that four qualified engineers do not agree on the categories either. That discovery is the deliverable.

The data — real maintenance short text

Corpus	MaintIE — maintenance work-order short texts, expert-annotated, released under the MIT licence by the NLP-TLP group at the University of Western Australia.
Size	1,076 expert-annotated texts (gold) plus 7,000 coarse-annotated (silver). You need 200.
Character	Genuine engineer-written fragments — abbreviations, missing verbs, inconsistent terminology. Exactly the ambiguity that makes this task honest.
Source	github.com/nlp-tlp/maintie → <code>data/gold_release.json</code> . Facilitator has a copy on the share if VM egress is blocked.

```
git clone https://github.com/nlp-tlp/maintie.git
python -c "import json,random,pandas as pd; random.seed(11); \
d=json.load(open('maintie/data/gold_release.json')); s=random.sample(d,200); \
pd.DataFrame({'id':range(200),'text':[' '.join(x['tokens']) for x in s]}).to_csv('wo200.csv',index=False)"
```

Step 1 · Agree the taxonomy — before you look at the data

As a team, write down eight categories and a one-line definition of each. Do it now, from engineering knowledge, before reading a single record. Suggested starting point, to be argued with rather than accepted: *seal or leak · bearing or rotating element · electrical and control · lubrication and oil system · cooling and heat exchange · filtration and blockage · fastening, mounting and alignment · instrumentation and sensing*. Add "**unclear**" as a ninth and treat its size as a diagnostic.

Step 2 · Classify independently — the moment the lab turns

Each of the four team members classifies **the same first 40 records**, alone, no discussion. Then compute agreement.

- **Fleiss' kappa** across all four annotators, and pairwise **Cohen's kappa**.
- The **disagreement matrix** — which category pairs are confused, and how often.
- The "**unclear**" **rate**. Above roughly 15% and your taxonomy is the problem, not the text.

Kappa	What it means for anything you build on this data
Below 0.20	The task definition needs work before any modelling continues.
0.21 – 0.40	Fair. Expect a low ceiling and say so in the project plan.
0.41 – 0.60	Moderate — usable, but the definitions would repay an hour of attention. Most teams land here.
0.61 – 0.80	Substantial. Usable ground truth.
Above 0.80	Excellent, and rare on genuinely subjective industrial text.

The sentence this step earns you. "Our four engineers agree at kappa 0.52. Any classifier trained on this labelling has a realistic ceiling around that level, so a 90% accuracy target is not ambitious — it is incoherent, because it asks the model to be more consistent than the ground truth it learns from." Put that on the dashboard. It is the strongest single argument in the entire Day 1 assignment.

Step 3 · Let the model classify, then measure it as an annotator

Use Azure OpenAI to classify all 200 records into your taxonomy, returning strict JSON: `{id, category, confidence, evidence_span}`. Then compute **kappa between the model and your human consensus on the 40 records the humans labelled**.

Define consensus before you compute anything: a record's consensus label is the strict majority of the four annotators. A 2–2 split has **no** consensus — mark it **contested**, exclude it from the kappa calculation, and report the contested count as a headline number. On a task at kappa ≈ 0.5 you should expect several. Those records are not a nuisance to be resolved; they are the clearest evidence you will produce all day that the category boundary does not exist yet.

COPILOT — BUILD THE CLASSIFIER AND ITS SCORER TOGETHER

Write `classify.py`. For each row in `wo200.csv` call Azure OpenAI with a system prompt containing our nine category definitions verbatim. Force JSON output: `{"id":int,"category":str,"confidence":float,"evidence_span":str}`. The category must be one of the nine exactly; if the text does not clearly support one, return "unclear" rather than guessing. Batch 20 records per call, retry once on rate limit, and log every raw response to disk. Then write `score_agreement.py` computing Cohen's kappa between model labels and human consensus on the 40 overlapping records, plus the confusion matrix between them. Do not report accuracy anywhere.

Watch for — the trap in this task. The model will return clean, confident, well-formatted categories for all 200 records, including the ones your four engineers could not agree on. Confidence is not agreement. Look specifically at the records where humans split: that is where the model's answer is least trustworthy and most likely to be accepted without challenge, because it arrives without hesitation. An annotator that never says "unclear" on genuinely unclear text is not a good annotator — it is a fast one.

Step 4 · Count, then weight by cost

Build two Pareto charts side by side.

Chart	Ranks by
Frequency Pareto	Number of work orders per category, with the cumulative line.
Cost-weighted Pareto	Frequency $\times$ business cost per occurrence, with the cumulative line.

Use these cost weights, and state them on the dashboard:

Category	Cost per occurrence	Why
Bearing / rotating element	€ 6,800	Usually an unplanned stoppage and a long repair
Electrical and control	€ 3,100	Diagnosis time dominates
Cooling and heat exchange	€ 2,400	Often escalates if missed
Seal or leak	€ 1,500	Frequent, contained, quick
Lubrication / oil system	€ 1,200	Mostly consumable replacement
Fastening, mounting, alignment	€ 900	Quick, but recurs
Instrumentation and sensing	€ 800	Low cost, high nuisance value
Filtration and blockage	€ 600	Routine
<i>Unclear</i>	<i>not costed</i>	Report as a count, never as a cost. A large unclear bucket is a defect in your taxonomy, and hiding it inside a euro figure conceals exactly the thing you need to see.

The finding almost every team gets. The two charts do not have the same top two. The most frequent category is rarely the most expensive one, and a backlog built from raw counts sends the engineering effort to the wrong place. Frequency alone misleads; frequency × impact is the ranking that survives contact with a budget conversation.

Step 5 · Slice, then estimate headroom

Slice the counts by at least two dimensions available in or derivable from the text — equipment type, activity verb, whether the record mentions a replacement, record length as a proxy for diagnostic difficulty. Then, for the top three categories, estimate **headroom**: if this category were eliminated entirely, what would it be worth per year at your assumed volume? State the assumption.

Step 6 · The ranked hypothesis backlog

Convert the analysis into experiments. Five rows minimum. Score each as **expected gain × confidence ÷ effort**.

Hypothesis	Expected gain	Confidence	Effort	Priority
<i>Worked example:</i> Rewriting the definitions for the two most-confused categories raises inter-annotator kappa from 0.52 to 0.70	+0.18 kappa	High	3 days	Highest

Your five go here. At least one must be a labelling or definition hypothesis, at least one a data-collection hypothesis, and at most one a modelling hypothesis. If your list is all modelling, you have not understood Module 3.

Acceptance criteria

- ☐ Taxonomy was written before the data was read, and the "unclear" rate is reported.
- ☐ Fleiss' kappa across four annotators is on the dashboard, with the disagreement matrix.
- ☐ Model-versus-human kappa is reported — not model accuracy.
- ☐ Both Pareto charts are shown and the difference between them is stated in one sentence.
- ☐ Headroom is quantified for the top three categories with the volume assumption stated.
- ☐ Backlog has five scored rows, at most one of which is a modelling change.

Defend your result

"Our four engineers agree at kappa _____. Ranked by count the top problem is ____; ranked by cost it is _____. We would work on _____ first, worth roughly € _____ a year, and we would not work on _____ despite it being the most common."

Task 4 — Production Readiness Review

Day 1 assignment · scored

MODULE 2B

MCNEMAR

POWER

PEEKING

CALIBRATION

DRIFT

The scenario

Your Task 2 model is System A. (If you missed Task 2, a serviceable System A is a gradient-boosted tree on the raw AI4I sensor columns — quick to produce with Copilot.) Overnight, you will make exactly one change to it and call the result System B — and then you will do the thing almost no proof of concept does: **prove, properly, whether B is actually better**, and produce a review a release board could sign.

This task delivers all seven items on the Day 1 assignment slide. Work in your own repository; the console runs on the same VM.

Pre-registration — do this first and commit it before you run anything. Create `PREREGISTRATION.md` naming: the one primary metric, the exact pass criterion as a number, the statistical test you will use, and the result that would make you abandon the change. Commit it. The timestamp is part of your submission. Everything after this point is evaluated against that file, and a team that quietly changes the primary metric later will be found out by their own git history — which is exactly how it should work.

Part 1 · One change, paired properly

Choose one change: derived physical features, class weighting, threshold recalibration, or a different model family. One. Then evaluate A and B **on the same held-out items** and build the paired comparison.

Analysis	What to produce
McNemar's test	The 2×2 table of per-item outcomes. Report b and c , the discordant counts, not just the p-value. If $b + c$ is small, say so plainly — the two systems rarely disagree, no test will find a difference, and that is itself the finding.
Permutation test	10,000 shuffles of the per-item outcome assignment, giving a p-value that assumes nothing about the distribution.
Effect size	Absolute difference first. Relative difference second, and only alongside the absolute. Number needed — how many machines must be scored for one extra failure caught — is the version a service manager will actually use.
Paired bootstrap	95% interval on the <i>difference</i> . Does it contain zero?

Watch for. An unpaired two-sample test on the same evaluation set is the most common statistical error in AI work. Items differ in difficulty; unpaired comparison treats that variation as noise, and the noise is usually larger than the improvement you are looking for. Pair, and item difficulty cancels.

Part 2 · Power — could you have detected it at all?

Compute the sample size needed to detect a 5-point, a 10-point and a 20-point improvement at $\alpha = 0.05$, power = 0.80. Compare against the number of positives in your test set. Most teams discover their evaluation set can only ever confirm large, obvious wins.

Improvement to detect	Approx. n per group	Note
0.60 → 0.80 (20 points)	≈ 90	Large effects are cheap to confirm
0.60 → 0.70 (10 points)	≈ 360	Four times the sample for half the effect
0.60 → 0.65 (5 points)	≈ 1,500	Again roughly four times

Figures are indicative and convey the scaling relationship — halving the effect roughly quadruples the sample.

Watch for — this table is for the wrong design, deliberately. Those are sample sizes for two *independent* groups, and Part 1 told you to pair. Under McNemar the power depends only on the **discordant pairs** — the items where exactly one system is right — so the quantity to compute is not "n per group" but $n_{\text{discordant}} \approx (z_{\alpha/2} + z_{\beta})^2 / (p_b - p_c)^2 \times (p_b + p_c)$. Count your own b and c, then work out how many *more* evaluation items you would need. Most teams find their gold set is an order of magnitude too small, and that finding is worth more than the comparison it invalidates.

Part 3 · The peeking simulator — fifteen lines, and the most persuasive thing you will build

Simulate two **identical** systems. Evaluate after every 10 accumulated test items and stop the moment $p < 0.05$. Repeat 1,000 times and report how often you declared a winner when none existed.

What you will see. The nominal 5% false positive rate climbs steeply — with frequent enough checking it can pass 30%. Show the curve of false positive rate against checking frequency on your console. Then state the fix in one line: fix the sample size and the stopping rule in advance, or use a sequential design that accounts for planned interim looks. This is the cheapest, most persuasive statistics demonstration in the whole programme, and it is the cheapest thing in this pack to build.

Part 4 · Calibration and drift

- **Reliability curve** — predicted probability against observed frequency, ten bins, plus the Brier score. If the model says 0.8 and is right 55% of the time, every threshold rule built on it behaves unexpectedly.
- **Drift** — split the data into an early and a late window along a defensible ordering (tool-wear band or record order). Run a Kolmogorov–Smirnov test and a Population Stability Index on each feature, and re-measure performance on the late window.
- **Name the type.** Covariate shift, label shift or concept drift — and state the response each demands. They are not interchangeable.
- **The retraining trigger** — the specific, numeric condition that would make you retrain. "When PSI on any top-five feature exceeds 0.25, or late-window recall drops more than one interval width below baseline."

Part 5 · The readiness console and the verdict

One page, five sections, ending in a single verdict: **SHIP · SHIP WITH LIMITS · DO NOT SHIP.**

Section	Contents
Baseline and metric set	Three to five metrics with targets, evaluation set described and versioned.
The comparison	McNemar table, permutation p, effect size, paired interval, and the pre-registered criterion beside the result.
Power and honesty	Detectable effect at your sample size; peeking curve.
Calibration and drift	Reliability curve, Brier, PSI/KS table, drift type, retraining trigger.
Operating point and authority	Threshold in alerts per engineer per day, expected cost, and an action-authority matrix : what the system does automatically, what requires human approval, what it is forbidden to do.

COPILOT — THE READINESS REPORT GENERATOR

Write `readiness_report.py`. Assemble every computed result into a single facts JSON — no prose, only numbers, intervals, test statistics, thresholds and the pre-registered criterion. Pass it to Azure OpenAI with this system prompt: "You are writing a one-page production readiness review for an engineering release board at a manufacturing company. Use only the supplied facts. Never state a number that is not in the facts. Every performance claim must carry its interval. State the verdict as exactly one of SHIP, SHIP WITH LIMITS, DO NOT SHIP, and give the single strongest reason against your own verdict."
Render the result to HTML with `jinja2`, then to PDF with `fpdf2` — or print the HTML from the browser if the VM has no PDF toolchain. The format is not scored; the content is.

Deeper detail — why the report must argue against itself. Requiring the strongest counter-argument in the same document is not a rhetorical flourish. It is the cheapest available defence against the failure mode that ends most proofs of concept: a review that reads only as advocacy, signed by people who had no way to know what was omitted. If your model cannot produce a real counter-argument from the facts you gave it, that usually means you did not measure enough.

Submission

- ☐ `PREREGISTRATION.md` , committed before any result was computed.
- ☐ Running console URL.
- ☐ Generated readiness report PDF, with the verdict and the counter-argument.
- ☐ Git log serving as experiment lineage — one commit per configuration, with metrics in the message.
- ☐ Completed seven-point leakage checklist, answered honestly for your own pipeline.

Defend your result

Present the verdict. A second team is nominated to attack it and must open with the question: *"what would have made you decide the other way?"* Teams are scored on the quality of the answer, not the popularity of the verdict. **A well-defended DO NOT SHIP outscores a poorly-evidenced SHIP.**

Where the data comes from

Item	Reference
AI4I 2020 Predictive Maintenance Dataset Task 2, Task 4	UCI Machine Learning Repository, dataset 601. 10,000 rows, roughly 3.4% positive class; five failure modes TWF, HDF, PWF, OSF, RNF, of which RNF is random by construction. archive.ics.uci.edu/dataset/601
MaintIE maintenance short-text corpus Task 3	Bikaun et al., LREC-COLING 2024. 1,076 expert-annotated and 7,000 coarse-annotated maintenance texts, MIT licence. github.com/nlp-tlp/maintie
MetroPT-3 Dataset optional extension	UCI dataset 791. The Air Production Unit compressor of a metro train — 1,516,948 readings at 1 Hz across 15 sensors, February to August 2020, with four documented air-leak failures. The closest public analogue to compressed-air equipment; use it if you want a real time-series lead-time exercise. archive.ics.uci.edu/dataset/791
Control-chart run-length figures Task 1	Champ & Woodall, <i>Technometrics</i> 29(4), 1987. Plain 3σ chart $ARL_0 = 370.4$; the supplementary runs rules reduce it to approximately 225, 166 and 153 individually, and to about 92 in combination with the 3σ rule.
Runs rules Task 1	Nelson, <i>Journal of Quality Technology</i> , 1984 — the eight rules used in Task 1.

On data hygiene. Nothing in this pack requires Atlas Copco data, and no customer or production data should be placed on a training virtual machine. The scenarios are framed in Atlas Copco terms — compressor elements, installed-base service, field engineers — because the framing is what makes the statistics stick. The data itself is public or synthetic throughout.

Every cost figure here is a placeholder

The euro values in these tasks — €180 for a line stop, €9,500 for an unplanned stoppage, €6,800 for a bearing failure — are illustrative. They are plausible, and they are not yours. Every threshold you selected today, and every ranking in your hypothesis backlog, moves when those numbers change.

The first thing worth doing on Monday. Find the real figures for one system you actually work on: what a false alarm costs, what a missed detection costs, and how many alerts the people receiving them will read in a day. Then re-run the threshold and cost analysis from Tasks 1 and 2 with those numbers in place. That single substitution converts everything in this pack from an exercise into a decision your organisation can act on, and it is usually a single sitting.

The five sentences worth taking out of the room

- 1 A single accuracy number is not a specification. Every system needs its own metric set with targets, defined before improvement work begins.
- 2 You cannot improve what you cannot measure honestly. The evaluation set and its confidence intervals come first; everything else depends on them.
- 3 State the hypothesis and the pass criterion before the experiment — and check whether your sample size can detect the effect you are hoping for.
- 4 Find the failure pattern before reaching for more data. Two or three root causes usually account for most of the loss.
- 5 Threshold and calibration work is nearly free, and is often the largest single gain available.

Before you leave today

Name, as a team, the single biggest thing you learned about your own proof of concept today. Write it down. Most teams find that what they name turns out to be an architecture problem — which is where Day 2 begins.